AI-102 Natural Language Processing (NLP)

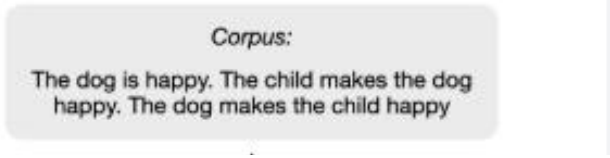
Dr. Jimmy Ardiansyah

EMERGING & COLLABORATIVE STUDIES

Feature Engineering for NLP

Bag of Words (BoW)

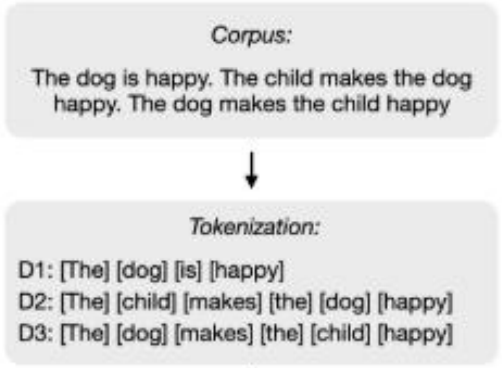
"Bag of Words" (BoW) is a fundamental method used in natural language processing (NLP) for text representation. It converts text into numerical features by treating each document as an unordered collection of words, ignoring grammar, word order, and context, but retaining the frequency of each word. it involves three main steps



Bag of Words (BoW)

"Bag of Words" (BoW) is a fundamental method used in natural language processing (NLP) for text representation. It converts text into numerical features by treating each document as an unordered collection of words, ignoring grammar, word order, and context, but retaining the frequency of each word. it involves three main steps

Tokenizing the Text : This is the initial step where text is broken down into individual words or "tokens." For example, the sentence "*'The dog is happy. The child makes the dog happy. The dog makes the child happy'*" would be tokenized into [“The", “Dog", "is", “happy"]. Tokenization is crucial for identifying the basic units that will form the vocabulary.

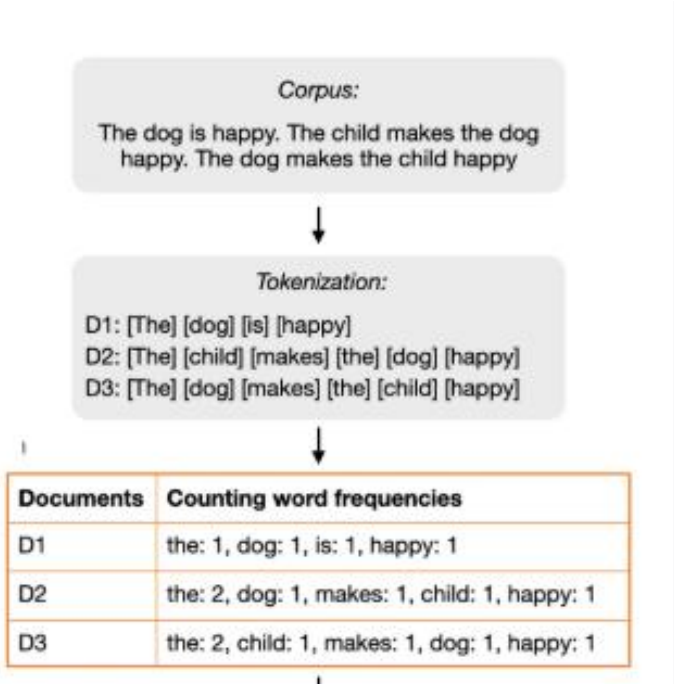


Bag of Words (BoW)

"Bag of Words" (BoW) is a fundamental method used in natural language processing (NLP) for text representation. It converts text into numerical features by treating each document as an unordered collection of words, ignoring grammar, word order, and context, but retaining the frequency of each word. it involves three main steps

Tokenizing the Text : This is the initial step where text is broken down into individual words or "tokens." For example, the sentence "*'The dog is happy. The child makes the dog happy. The dog makes the child happy'*" would be tokenized into [“The", “Dog", "is", “happy"]. Tokenization is crucial for identifying the basic units that will form the vocabulary.

Building a Vocabular: After tokenization, a unique set of words from the entire text corpus is compiled to create the vocabulary.



Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

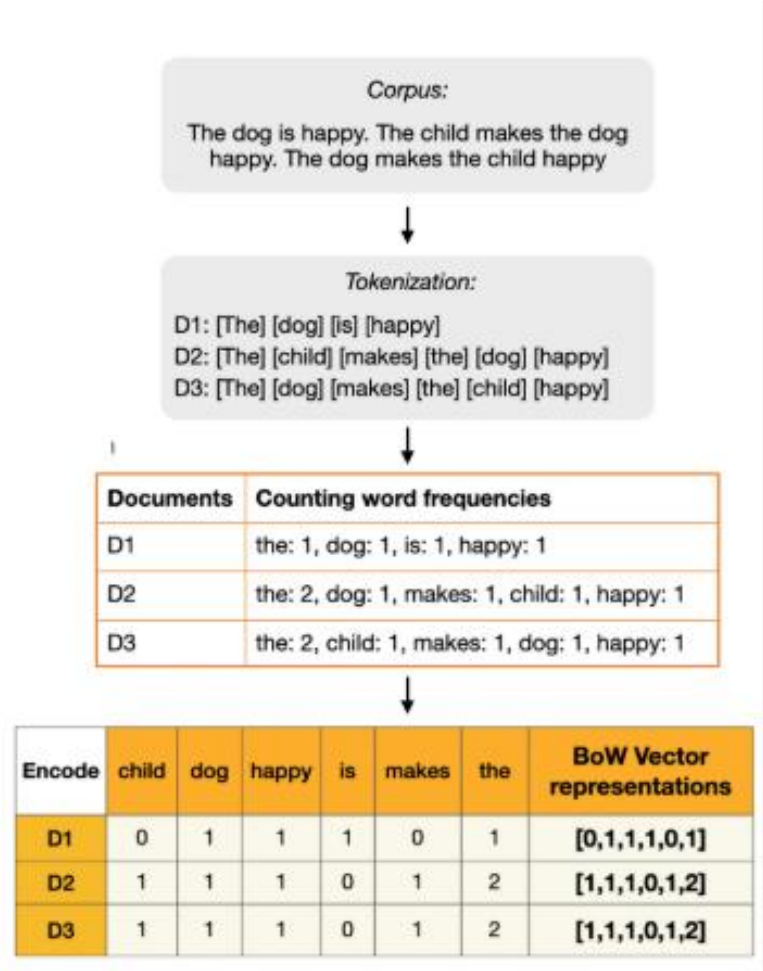
Bag of Words (BoW)

"Bag of Words" (BoW) is a fundamental method used in natural language processing (NLP) for text representation. It converts text into numerical features by treating each document as an unordered collection of words, ignoring grammar, word order, and context, but retaining the frequency of each word. it involves three main steps

Tokenizing the Text : This is the initial step where text is broken down into individual words or "tokens." For example, the sentence "*'The dog is happy. The child makes the dog happy. The dog makes the child happy'*" would be tokenized into [“The", “Dog", "is", “happy"]. Tokenization is crucial for identifying the basic units that will form the vocabulary.

Building a Vocabular: After tokenization, a unique set of words from the entire text corpus is compiled to create the vocabulary.

Vectorizing the Text: In this final step, each document is converted into a numerical vector based on the established vocabulary.



Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

Advantages and Limitation Bag of Words

Advantages and Limitation Bag of Words

- Simplicity: BoW is "remarkably straightforward and easy to understand," making it highly accessible and allowing for "quick adoption and experimentation.“
- Efficiency: It offers "computational efficiency," especially for "small to medium-sized text corpora," leading to faster processing times.
- Baseline: BoW often serves as a "robust baseline for more complex models," providing a foundational benchmark for comparison with advanced techniques.

Advantages and Limitation Bag of Words

- Simplicity: BoW is "remarkably straightforward and easy to understand," making it highly accessible and allowing for "quick adoption and experimentation.“
- Efficiency: It offers "computational efficiency," especially for "small to medium-sized text corpora," leading to faster processing times.
- Baseline: BoW often serves as a "robust baseline for more complex models," providing a foundational benchmark for comparison with advanced techniques.

- Loss of Context: A significant drawback is its "neglect of the order and context of words." This means "the sequence of words often plays a crucial role in conveying the true meaning of the text," which BoW fails to capture, potentially missing "important nuances.“
- High Dimensionality: As the vocabulary size increases with larger text corpora, BoW can lead to "high-dimensional feature vectors," making the model "cumbersome and difficult to manage.“
- Sparsity: The generated feature vectors are often "sparse," meaning "most elements in these vectors are zero." This "can be inefficient to process and may require additional computational resources and techniques to handle effectively."

Advantages and Limitation Bag of Words

```python
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

# 1. Sample Text Data
# A simple list of documents (sentences) for our demonstration.
documents = [
    "The quick brown fox jumps over the lazy dog.",
    "A lazy cat naps on the warm sun.",
    "The sun is warm and bright."
]

# 2. Create the Bag of Words Model
# CountVectorizer handles tokenization, building the vocabulary, and counting word
# It automatically converts text to lowercase and removes punctuation.
# Stop words (common words like "the", "a", "is") are optional but often useful to
vectorizer = CountVectorizer(stop_words='english')

# 3. Fit and Transform the Data
# 'fit' learns the vocabulary from the documents.
# 'transform' converts the documents into a matrix of token counts.
bow_matrix = vectorizer.fit_transform(documents)

# 4. Inspect the Output
# Get the vocabulary (unique words) that the model learned.
vocabulary = vectorizer.get_feature_names_out()

# Convert the sparse matrix to a dense DataFrame for easier viewing and interpreta
# This makes the word counts for each document visible.
bow_df = pd.DataFrame(bow_matrix.toarray(), columns=vocabulary)

# Print the results
print("--- Our Vocabulary (Learned from the documents) ---")
print(vocabulary)
print("\n--- The Bag of Words Matrix (Document-Term Matrix) ---")
print(bow_df)
print("\n--- Shape of the matrix (documents x words) ---")
print(bow_df.shape)
```

```
--- Our Vocabulary (Learned from the documents) ---
['bright' 'brown' 'cat' 'dog' 'fox' 'jumps' 'lazy' 'naps' 'quick' 'sun'
 'warm']

--- The Bag of Words Matrix (Document-Term Matrix) ---
   bright  brown  cat  dog  fox  jumps  lazy  naps  quick  sun  warm
0       0      1    0    1    1      1     1     0      1    0     0
1       0      0    1    0    0      0     1     1      0    1     1
2       1      0    0    0    0      0     0     0      0    1     1

--- Shape of the matrix (documents x words) ---
(3, 11)
```

TF-IDF (Term Freq.-Inverse Document Freq.)

TF-IDF is a numerical statistic used in NLP to reflect the importance of a word to a document within a corpus. Unlike the Bag of Words model, which only counts word occurrences, TF-IDF provides a more sophisticated representation by considering the significance of each word in relation to the entire text collection.

TF-IDF (Term Freq.-Inverse Document Freq.)

TF-IDF is a numerical statistic used in NLP to reflect the importance of a word to a document within a corpus. Unlike the Bag of Words model, which only counts word occurrences, TF-IDF provides a more sophisticated representation by considering the significance of each word in relation to the entire text collection.

Term Frequency (TF): This is the simple part—just the count of how many times a word shows up in a single document. A word like "apple" appearing 5 times in a document has a higher TF than a word like "banana" that only appears once.

$$TF(t,d) = \frac{\text{Number of times } t \text{ appears in } d}{\text{Total number of terms in } d}$$

TF-IDF (Term Freq.-Inverse Document Freq.)

TF-IDF is a numerical statistic used in NLP to reflect the importance of a word to a document within a corpus. Unlike the Bag of Words model, which only counts word occurrences, TF-IDF provides a more sophisticated representation by considering the significance of each word in relation to the entire text collection.

Term Frequency (TF): This is the simple part—just the count of how many times a word shows up in a single document. A word like "apple" appearing 5 times in a document has a higher TF than a word like "banana" that only appears once.

$$TF(t,d) = \frac{\text{Number of times } t \text{ appears in } d}{\text{Total number of terms in } d}$$

Inverse Document Frequency (IDF): This is the clever part. It measures how unique or rare a word is across all the documents.

- Common words like "the," "is," or "and" appear everywhere. Their IDF score will be very low, so they don't get much importance.
- Rare words like "photosynthesis" or "quantum" are very specific. They will have a high IDF score, which makes them more important.

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing } t}\right)$$

TF-IDF (Term Freq.-Inverse Document Freq.)

TF-IDF is a numerical statistic used in NLP to reflect the importance of a word to a document within a corpus. Unlike the Bag of Words model, which only counts word occurrences, TF-IDF provides a more sophisticated representation by considering the significance of each word in relation to the entire text collection.

Term Frequency (TF): This is the simple part—just the count of how many times a word shows up in a single document. A word like "apple" appearing 5 times in a document has a higher TF than a word like "banana" that only appears once.

$$TF(t,d) = \frac{\text{Number of times } t \text{ appears in } d}{\text{Total number of terms in } d}$$

Inverse Document Frequency (IDF): This is the clever part. It measures how unique or rare a word is across all the documents.

- Common words like "the," "is," or "and" appear everywhere. Their IDF score will be very low, so they don't get much importance.
- Rare words like "photosynthesis" or "quantum" are very specific. They will have a high IDF score, which makes them more important.

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing } t}\right)$$

TF-IDF Score is the result of multiplying these two scores together: TF x IDF.

$$TF - IDF(t,d) = TF(t,d) \times IDF(t)$$

Advantages and Limitation TF-IDF

Advantages and Limitation TF-IDF

Importance Weighting: TF-IDF assigns a higher value to words that are significant to a specific document, while downplaying the importance of common, noisy words like "the" or "is." This highlights the unique vocabulary of each document.

Improved Feature Representation: Unlike basic frequency counts, TF-IDF provides a more nuanced and informative representation of text. It balances how often a word appears in a document (Term Frequency) with how rare it is across the entire collection of documents (Inverse Document Frequency).

Versatility: TF-IDF can be applied to a wide range of NLP tasks, including information retrieval, text classification, and clustering. The numerical data it generates can be easily used as input for various machine learning models.

Enhanced Performance: By providing a better representation of text, TF-IDF often improves the performance of machine learning models in classification and clustering tasks, especially when dealing with large datasets where distinguishing important terms is critical.

Ease of Use: TF-IDF is easy to implement using popular Python libraries like scikit-learn, making it accessible for both practitioners and researchers.

Advantages and Limitation TF-IDF

Importance Weighting: TF-IDF assigns a higher value to words that are significant to a specific document, while downplaying the importance of common, noisy words like "the" or "is." This highlights the unique vocabulary of each document.

Improved Feature Representation: Unlike basic frequency counts, TF-IDF provides a more nuanced and informative representation of text. It balances how often a word appears in a document (Term Frequency) with how rare it is across the entire collection of documents (Inverse Document Frequency).

Versatility: TF-IDF can be applied to a wide range of NLP tasks, including information retrieval, text classification, and clustering. The numerical data it generates can be easily used as input for various machine learning models.

Enhanced Performance: By providing a better representation of text, TF-IDF often improves the performance of machine learning models in classification and clustering tasks, especially when dealing with large datasets where distinguishing important terms is critical.

Ease of Use: TF-IDF is easy to implement using popular Python libraries like scikit-learn, making it accessible for both practitioners and researchers.

Sparsity: Like the Bag of Words model, TF-IDF can produce high-dimensional and sparse feature vectors, especially with large vocabularies. This can lead to increased memory usage and computational inefficiency.

Context Ignorance: A significant drawback is that TF-IDF treats each word independently. It does not capture semantic relationships, word order, or the surrounding context, which is crucial for a deeper understanding of language. More advanced models like word embeddings are designed to overcome this limitation.

TF-IDF Example

We'll use three short documents:

Document 1: "The cat sat on the mat."

Document 2: "The dog chased the cat."

Document 3: "The dog and the cat."

Step 1: Calculate Term Frequency (TF)

Term Frequency is just the count of each word in each document. We'll normalize this by dividing the word's count by the total number of words in that document.

Word	Doc 1 (6 words)	Doc 2 (5 words)	Doc 3 (5 words)
the	$2/6 = 0.33$	$2/5 = 0.40$	$2/5 = 0.40$
cat	$1/6 = 0.17$	$1/5 = 0.20$	$1/5 = 0.20$
sat	$1/6 = 0.17$	0	0
on	$1/6 = 0.17$	0	0
mat	$1/6 = 0.17$	0	0
dog	0	$1/5 = 0.20$	$1/5 = 0.20$
chased	0	$1/5 = 0.20$	0
and	0	0	$1/5 = 0.20$

Step 2: Calculate Inverse Document Frequency (IDF)

Inverse Document Frequency measures how rare a word is across all documents.

Word	Documents containing the word	IDF Score
the	3	$\log\_e(3/3) = \log\_e(1) = 0$
cat	3	$\log\_e(3/3) = \log\_e(1) = 0$
sat	1	$\log\_e(3/1) = \log\_e(3) \approx 1.10$
on	1	$\log\_e(3/1) = \log\_e(3) \approx 1.10$
mat	1	$\log\_e(3/1) = \log\_e(3) \approx 1.10$
dog	2	$\log\_e(3/2) = \log\_e(1.5) \approx 0.41$
chased	1	$\log\_e(3/1) = \log\_e(3) \approx 1.10$
and	1	$\log\_e(3/1) = \log\_e(3) \approx 1.10$

Step 3: Calculate TF-IDF Score

Finally, we multiply the TF and IDF scores for each word in each document

Word	Doc 1	Doc 2	Doc 3
the	$0.33 \times 0 = 0$	$0.40 \times 0 = 0$	$0.40 \times 0 = 0$
cat	$0.17 \times 0 = 0$	$0.20 \times 0 = 0$	$0.20 \times 0 = 0$
sat	$0.17 \times 1.10 \approx 0.19$	$0 \times 1.10 = 0$	$0 \times 1.10 = 0$
on	$0.17 \times 1.10 \approx 0.19$	$0 \times 1.10 = 0$	$0 \times 1.10 = 0$
mat	$0.17 \times 1.10 \approx 0.19$	$0 \times 1.10 = 0$	$0 \times 1.10 = 0$
dog	0	$0.20 \times 0.41 \approx 0.08$	$0.20 \times 0.41 \approx 0.08$
chased	0	$0.20 \times 1.10 \approx 0.22$	0
and	0	0	$0.20 \times 1.10 \approx 0.22$

Summary:

The final TF-IDF scores show that words like **"the"** and **"cat"** have **no importance (a score of 0)** because they are so common. However, words that are unique to a document, like "**chased**" or **"sat"**, get a much higher score, indicating they are more relevant and important for understanding the specific content of that document.

TF-IDF vs. Bag of Words

Feature	Bag of Words (BoW)	Term Frequency-Inverse Document Frequency (TF-IDF)
Core Method	Counts the occurrences of each word in a document.	Calculates a weight for each word based on its frequency in a document and its rarity across the entire corpus.
Word Importance	Treats all words equally. High-frequency common words (e.g., "the") can dominate the representation.	Assigns higher weights to words that are important to a specific document, downplaying common words.
Representation	Simple and easy to implement.	More nuanced and informative, providing a context-aware representation.
Information Value	Can be skewed by words that carry little useful information about the document's specific content.	Highlights terms that are more relevant to the specific context of the document, leading to more precise analysis.
Suitability	Best for basic text analysis where word importance is not a critical factor.	Better suited for scenarios where understanding the significance of terms within the broader corpus is crucial.

Word Embedding Techniques

Word embeddings are a powerful NLP technique that maps words to vectors of real numbers. The primary goal is to create a representation where words with similar meanings have similar vector representations. This provides a dense and more informative alternative to traditional models like Bag of Words and TF-IDF, which often fail to capture nuanced semantic relationships.

Word Embedding Techniques

Word embeddings are a powerful NLP technique that maps words to vectors of real numbers. The primary goal is to create a representation where words with similar meanings have similar vector representations. This provides a dense and more informative alternative to traditional models like Bag of Words and TF-IDF, which often fail to capture nuanced semantic relationships.

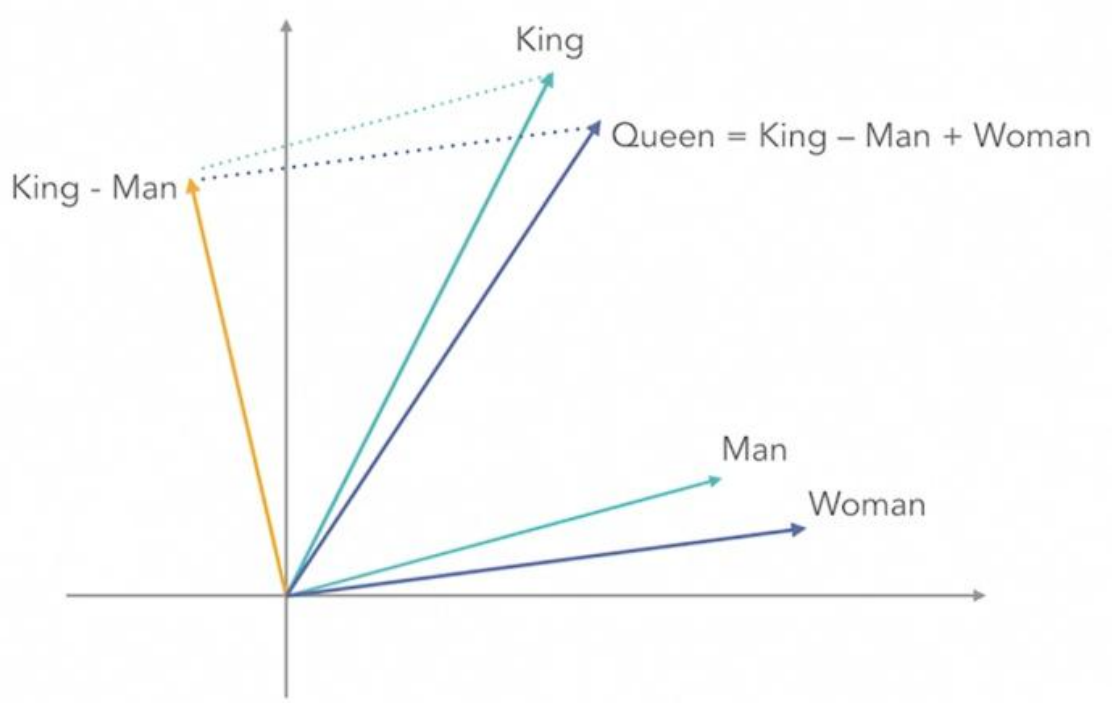
Semantic Similarity: Embeddings are designed so that words used in similar contexts are located near each other in the vector space. For example, the vectors for "king" and "queen" would be similar due to their frequent appearance in related contexts like royalty and governance.

Word Embedding Techniques

Word embeddings are a powerful NLP technique that maps words to vectors of real numbers. The primary goal is to create a representation where words with similar meanings have similar vector representations. This provides a dense and more informative alternative to traditional models like Bag of Words and TF-IDF, which often fail to capture nuanced semantic relationships.

Semantic Similarity: Embeddings are designed so that words used in similar contexts are located near each other in the vector space. For example, the vectors for "king" and "queen" would be similar due to their frequent appearance in related contexts like royalty and governance.

Continuous Vector Space: Words are represented as points in a continuous space, which allows for mathematical modeling of their relationships. A well-known example is the analogy *vector('king') - vector('man') + vector('woman')* resulting in a vector very close to *vector('queen')*.



Word Embedding Techniques

Word embeddings are a powerful NLP technique that maps words to vectors of real numbers. The primary goal is to create a representation where words with similar meanings have similar vector representations. This provides a dense and more informative alternative to traditional models like Bag of Words and TF-IDF, which often fail to capture nuanced semantic relationships.

Semantic Similarity: Embeddings are designed so that words used in similar contexts are located near each other in the vector space. For example, the vectors for "king" and "queen" would be similar due to their frequent appearance in related contexts like royalty and governance.

Continuous Vector Space: Words are represented as points in a continuous space, which allows for mathematical modeling of their relationships. A well-known example is the analogy *vector('king') - vector('man') + vector('woman')* resulting in a vector very close to *vector('queen')*.

Dimensionality Reduction: Compared to the very high-dimensional vectors created by methods like Bag of Words, embeddings provide a low-dimensional representation while preserving essential semantic information, making them more computationally efficient to process.

Word Embedding Techniques

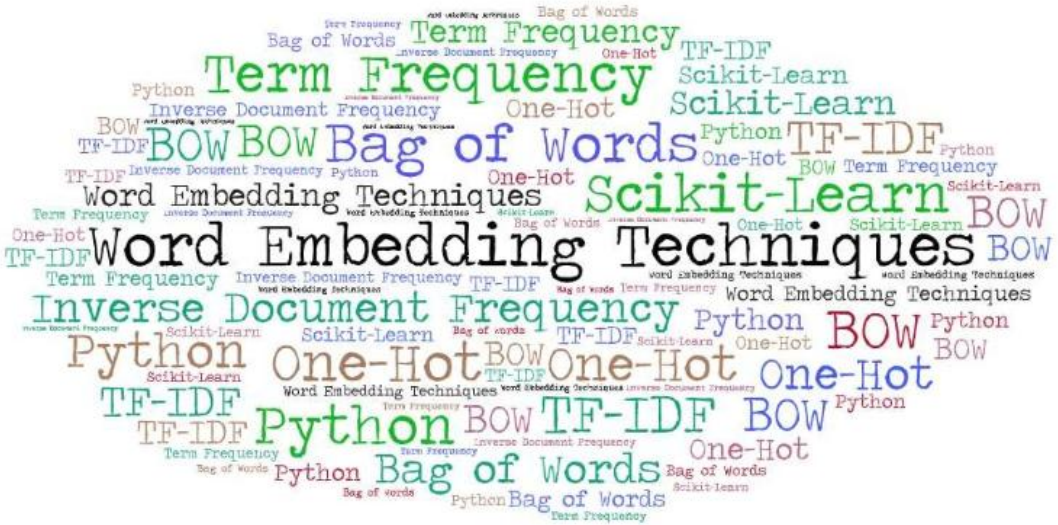
Word embeddings are a powerful NLP technique that maps words to vectors of real numbers. The primary goal is to create a representation where words with similar meanings have similar vector representations. This provides a dense and more informative alternative to traditional models like Bag of Words and TF-IDF, which often fail to capture nuanced semantic relationships.

Semantic Similarity: Embeddings are designed so that words used in similar contexts are located near each other in the vector space. For example, the vectors for "king" and "queen" would be similar due to their frequent appearance in related contexts like royalty and governance.

Continuous Vector Space: Words are represented as points in a continuous space, which allows for mathematical modeling of their relationships. A well-known example is the analogy *vector('king') - vector('man') + vector('woman')* resulting in a vector very close to *vector('queen')*.

Dimensionality Reduction: Compared to the very high-dimensional vectors created by methods like Bag of Words, embeddings provide a low-dimensional representation while preserving essential semantic information, making them more computationally efficient to process.

Transfer Learning: Pre-trained word embeddings, trained on massive text corpora, capture general linguistic properties. These can be applied to new NLP tasks (e.g., sentiment analysis, text classification) without needing to train a model from scratch, saving significant time and computational resources.



Word2Vec - Giving Words a Sense of Meaning

Word2Vec is a group of techniques in Natural Language Processing (NLP) used to create word embeddings. These are numerical representations of words (vectors) that capture their meaning and context. The core idea is that words that appear in similar contexts will have similar meanings and, therefore, will have similar vector representations.

Think of it like this: instead of a word being a standalone item, it becomes a point in a multi-dimensional space. Words with similar meanings will be located close to each other in that space. For example, the vector for "king" and the vector for "queen" would be very close, as would "man" and "woman." This allows computers to understand relationships between words.

Word2Vec works by using a simple neural network to predict words based on their neighbors. It has two main methods for doing this:

Word2Vec - Giving Words a Sense of Meaning

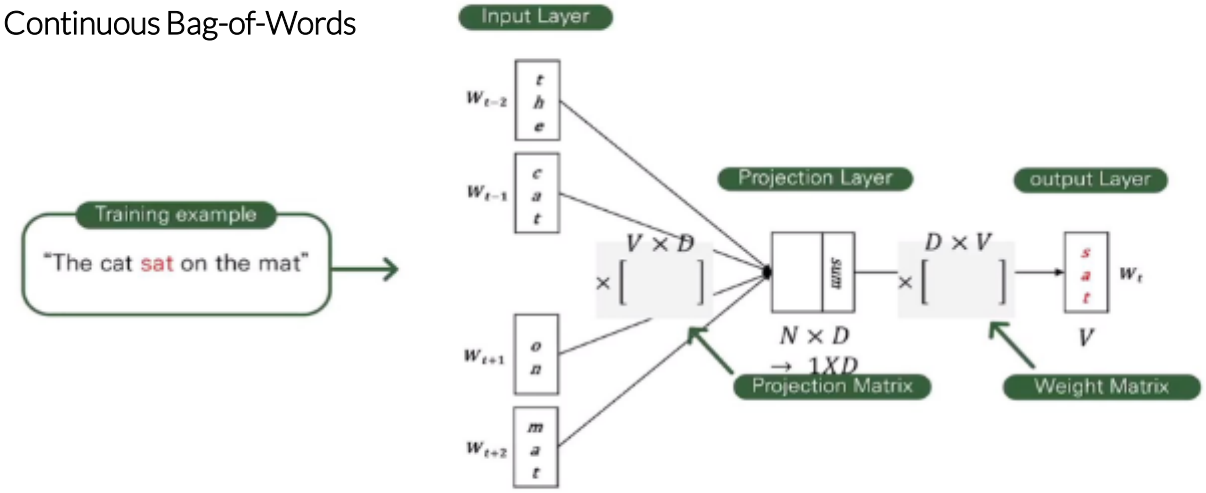
Word2Vec is a group of techniques in Natural Language Processing (NLP) used to create **word embeddings**. These are numerical representations of words (vectors) that capture their meaning and context. The core idea is that words that appear in similar contexts will have similar meanings and, therefore, will have similar vector representations.

Think of it like this: instead of a word being a standalone item, it becomes a point in a multi-dimensional space. Words with similar meanings will be located close to each other in that space. For example, the vector for "king" and the vector for "queen" would be very close, as would "man" and "woman." This allows computers to understand relationships between words.

Word2Vec works by using a simple neural network to predict words based on their neighbors. It has two main methods for doing this:

- **Continuous Bag-of-Words (CBOW):** This model predicts the current word based on its surrounding context words. For example, given the sentence **"The cat sat on the mat,"** the model might be trained to predict the word **"sat"** using the context words **"the," "cat," "on," and "mat."**

Continuous Bag-of-Words



Word2Vec - Giving Words a Sense of Meaning

Word2Vec is a group of techniques in Natural Language Processing (NLP) used to create **word embeddings**. These are numerical representations of words (vectors) that capture their meaning and context. The core idea is that words that appear in similar contexts will have similar vector representations.

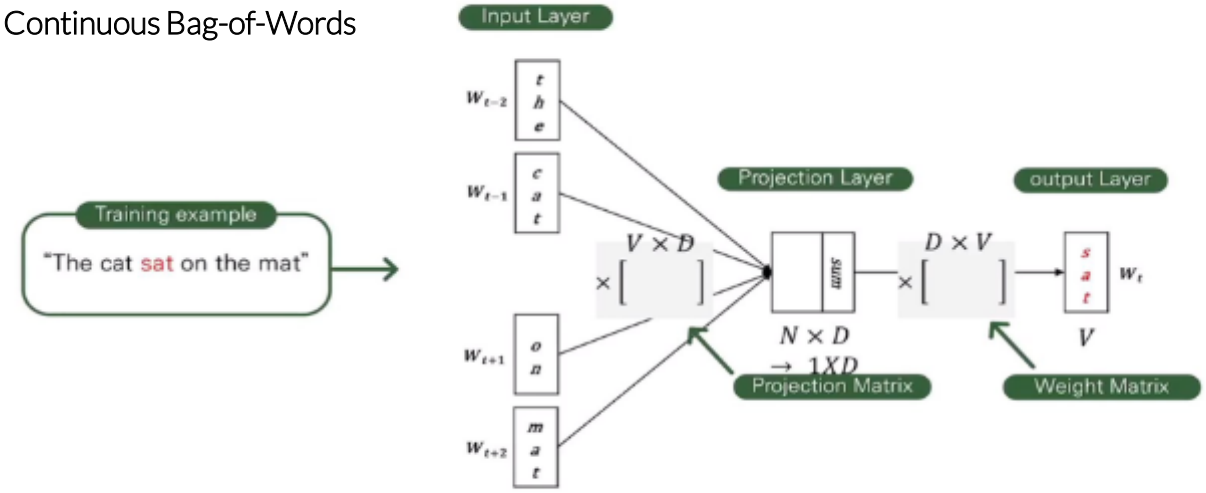
Think of it like this: instead of a word being a standalone item, it becomes a point in a multi-dimensional space. Words with similar meanings will be located close to each other in that space. For example, the vector for "king" and the vector for "queen" would be very close, as would "man" and "woman." This allows computers to understand relationships between words.

Word2Vec works by using a simple neural network to predict words based on their neighbors. It has two main methods for doing this:

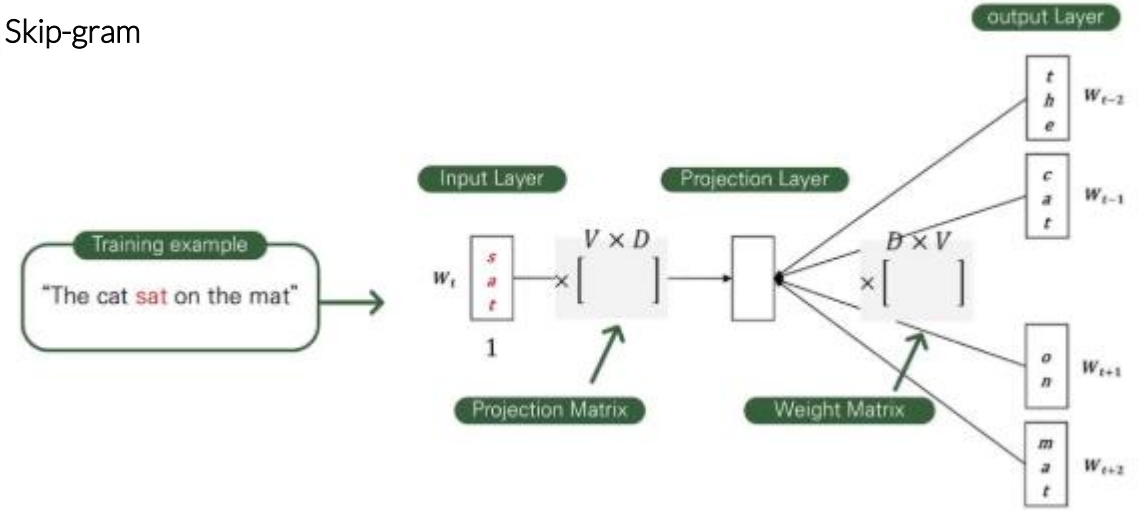
- **Continuous Bag-of-Words (CBOW):** This model predicts the current word based on its surrounding context words. For example, given the sentence **"The cat sat on the mat,"** the model might be trained to predict the word **"sat"** using the context words **"the," "cat," "on," and "mat."**

- **Skip-gram:** This model does the opposite. It takes a single word and tries to predict its surrounding context words. For example, given the word **"sat,"** the model might be trained to predict the words **"the," "cat," "on," and "mat."**

Continuous Bag-of-Words



Skip-gram



GloVe (Global Vectors for Word Representation)

GloVe, which stands for **Global Vectors for Word Representation**, is an unsupervised learning algorithm for generating **word embeddings**. It's similar to Word2Vec in its goal—to create numerical vectors that represent the meaning of words—but it uses a different approach. While Word2Vec focuses on local context (neighboring words), GloVe takes a **global approach** by analyzing the co-occurrence statistics of words across the entire corpus of text.

The core idea of GloVe is that the ratio of co-occurrence probabilities between words can be used to encode meaning. The model learns word vectors such that their dot product is related to the logarithm of the words' co-occurrence probability.

GloVe (Global Vectors for Word Representation)

GloVe, which stands for **Global Vectors for Word Representation**, is an unsupervised learning algorithm for generating **word embeddings**. It's similar to Word2Vec in its goal—to create numerical vectors that represent the meaning of words—but it uses a different approach. While Word2Vec focuses on local context (neighboring words), GloVe takes a **global approach** by analyzing the co-occurrence statistics of words across the entire corpus of text.

The core idea of GloVe is that the ratio of co-occurrence probabilities between words can be used to encode meaning. The model learns word vectors such that their dot product is related to the logarithm of the words' co-occurrence probability.

This method combines the best of two worlds:

- **Local Context Models :**These models learn from the local window of words.

- **Global Matrix Factorization Models:** These models, like Latent Semantic Analysis (LSA), analyze the entire dataset at once to find patterns.

GloVe creates a large co-occurrence matrix where each entry represents how many times two words appear together in the entire corpus. It then uses this matrix to learn vector representations for each word. The result is a set of word vectors that capture meaningful relationships, such as "king" minus "man" plus "woman" being approximately equal to "queen."

GloVe (Global Vectors for Word Representation)

GloVe, which stands for **Global Vectors for Word Representation**, is an unsupervised learning algorithm for generating **word embeddings**. It's similar to Word2Vec in its goal—to create numerical vectors that represent the meaning of words—but it uses a different approach. While Word2Vec focuses on local context (neighboring words), GloVe takes a **global approach** by analyzing the co-occurrence statistics of words across the entire corpus of text.

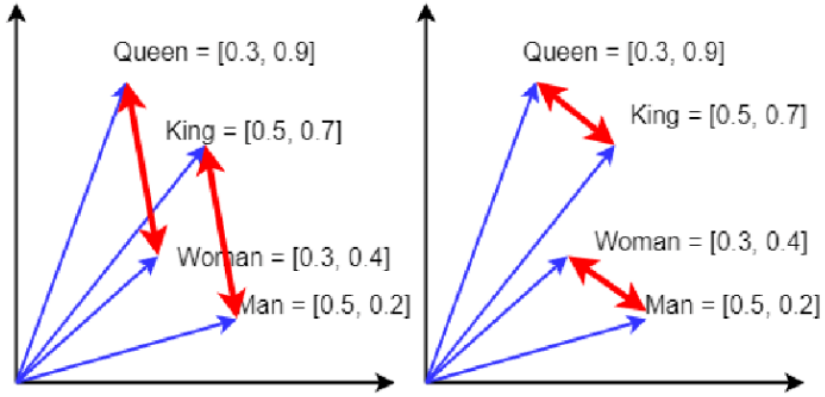
The core idea of GloVe is that the ratio of co-occurrence probabilities between words can be used to encode meaning. The model learns word vectors such that their dot product is related to the logarithm of the words' co-occurrence probability.

This method combines the best of two worlds:

- **Local Context Models :**These models learn from the local window of words.

- **Global Matrix Factorization Models:** These models, like Latent Semantic Analysis (LSA), analyze the entire dataset at once to find patterns.

GloVe creates a large co-occurrence matrix where each entry represents how many times two words appear together in the entire corpus. It then uses this matrix to learn vector representations for each word. The result is a set of word vectors that capture meaningful relationships, such as "king" minus "man" plus "woman" being approximately equal to "queen."



The image demonstrates the famous analogy: **King - Man + Woman ≈ Queen.**

King and Man: The red arrow pointing from "Man" to "King" represents the conceptual difference between the two. This vector captures the concept of "royalty" or "male authority."

GloVe (Global Vectors for Word Representation)

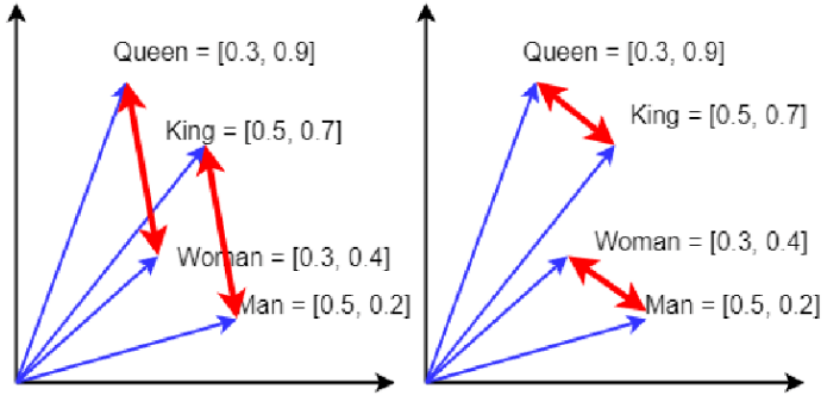
GloVe, which stands for **Global Vectors for Word Representation**, is an unsupervised learning algorithm for generating **word embeddings**. It's similar to Word2Vec in its goal—to create numerical vectors that represent the meaning of words—but it uses a different approach. While Word2Vec focuses on local context (neighboring words), GloVe takes a **global approach** by analyzing the co-occurrence statistics of words across the entire corpus of text.

The core idea of GloVe is that the ratio of co-occurrence probabilities between words can be used to encode meaning. The model learns word vectors such that their dot product is related to the logarithm of the words' co-occurrence probability.

This method combines the best of two worlds:

- **Local Context Models :**These models learn from the local window of words.
- **Global Matrix Factorization Models:** These models, like Latent Semantic Analysis (LSA), analyze the entire dataset at once to find patterns.

GloVe creates a large co-occurrence matrix where each entry represents how many times two words appear together in the entire corpus. It then uses this matrix to learn vector representations for each word. The result is a set of word vectors that capture meaningful relationships, such as "king" minus "man" plus "woman" being approximately equal to "queen."



The image demonstrates the famous analogy: **King - Man + Woman ≈ Queen.**

King and Man: The red arrow pointing from "Man" to "King" represents the conceptual difference between the two. This vector captures the concept of "royalty" or "male authority."

Woman and Queen: Similarly, the red arrow pointing from "Woman" to "Queen" represents the same abstract difference—the concept of "royalty."

GloVe (Global Vectors for Word Representation)

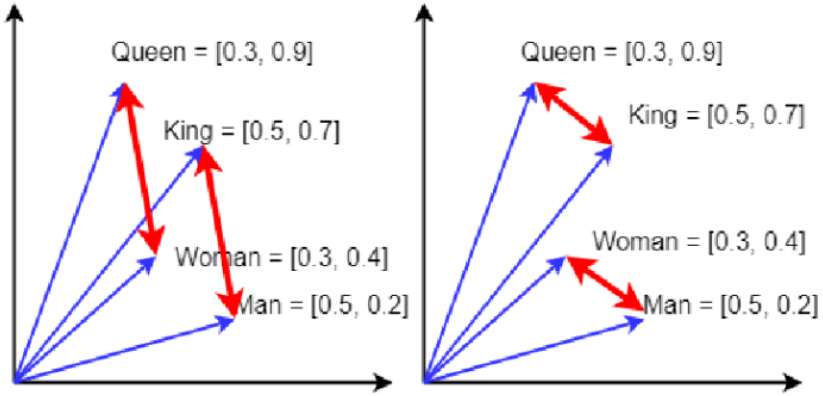
GloVe, which stands for **Global Vectors for Word Representation**, is an unsupervised learning algorithm for generating **word embeddings**. It's similar to Word2Vec in its goal—to create numerical vectors that represent the meaning of words—but it uses a different approach. While Word2Vec focuses on local context (neighboring words), GloVe takes a **global approach** by analyzing the co-occurrence statistics of words across the entire corpus of text.

The core idea of GloVe is that the ratio of co-occurrence probabilities between words can be used to encode meaning. The model learns word vectors such that their dot product is related to the logarithm of the words' co-occurrence probability.

This method combines the best of two worlds:

- **Local Context Models :**These models learn from the local window of words.
- **Global Matrix Factorization Models:** These models, like Latent Semantic Analysis (LSA), analyze the entire dataset at once to find patterns.

GloVe creates a large co-occurrence matrix where each entry represents how many times two words appear together in the entire corpus. It then uses this matrix to learn vector representations for each word. The result is a set of word vectors that capture meaningful relationships, such as "king" minus "man" plus "woman" being approximately equal to "queen."



The image demonstrates the famous analogy: **King - Man + Woman ≈ Queen**.

King and Man: The red arrow pointing from "Man" to "King" represents the conceptual difference between the two. This vector captures the concept of "royalty" or "male authority."

Woman and Queen: Similarly, the red arrow pointing from "Woman" to "Queen" represents the same abstract difference—the concept of "royalty."

Parallelism: The two red arrows are nearly parallel and have similar lengths. This visual similarity shows that the relationship between "King" and "Man" is very similar to the relationship between "Queen" and "Woman."

Therefore, by starting with the "King" vector, subtracting the "Man" vector (which removes the masculinity), and then adding the "Woman" vector (which adds femininity), you arrive at a new vector that is very close to the "Queen" vector. This works because the underlying word embeddings have learned to encode these abstract, relational meanings.

Word2Vec vs. GloVe

Feature	Word2Vec	GloVe (Global Vectors)
Core Method	Predicts words based on their immediate neighbors (local context). Uses neural networks (CBOW or Skip-Gram).	Performs matrix factorization on a global word-word co-occurrence matrix.
Context Focus	**Local:** Considers a limited window of words around a target word.	**Global & Local:** Captures statistical information from the entire corpus, considering both broad and immediate contexts.
Strengths	Highly efficient and fast to train on large datasets. Excels at capturing local contextual nuances.	Often produces more accurate embeddings for tasks benefiting from broader semantic relationships due to its global approach.

Introduction to BERT

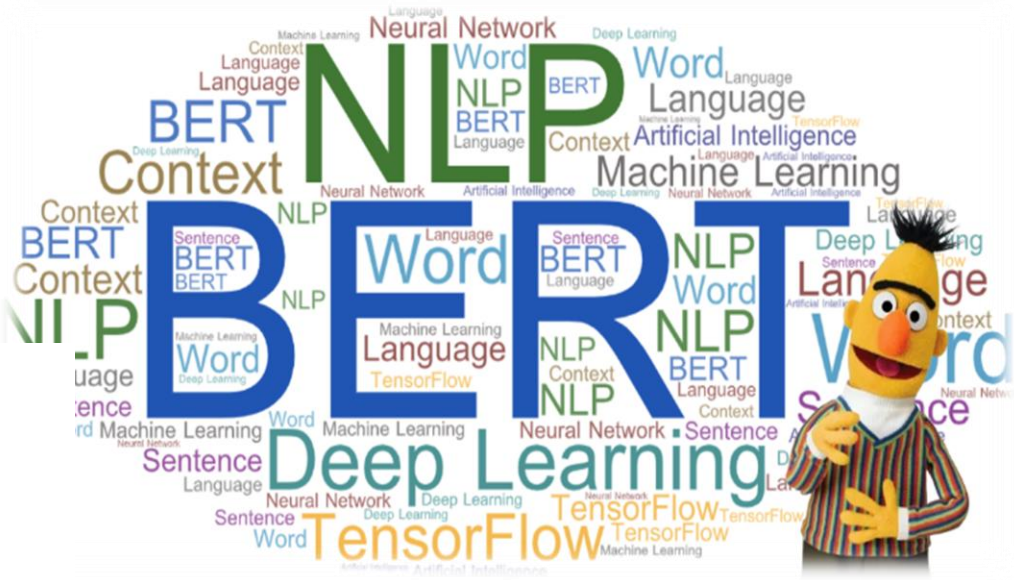
Earlier word embedding models like Word2Vec and GloVe created a single, static vector for each word. For example, the word "bank" would always have the same vector, regardless of its context. This is a problem because words often have multiple meanings.

Consider these two sentences:

"I need to go to the bank to deposit a check."

"The river bank overflowed after the heavy rain."

In the first sentence, "bank" refers to a financial institution, while in the second, it means the side of a river. Static embeddings fail to capture this **polysemy** (multiple meanings of a word), making them less effective for complex NLP tasks.



Introduction to BERT

Earlier word embedding models like Word2Vec and GloVe created a single, static vector for each word. For example, the word "bank" would always have the same vector, regardless of its context. This is a problem because words often have multiple meanings.

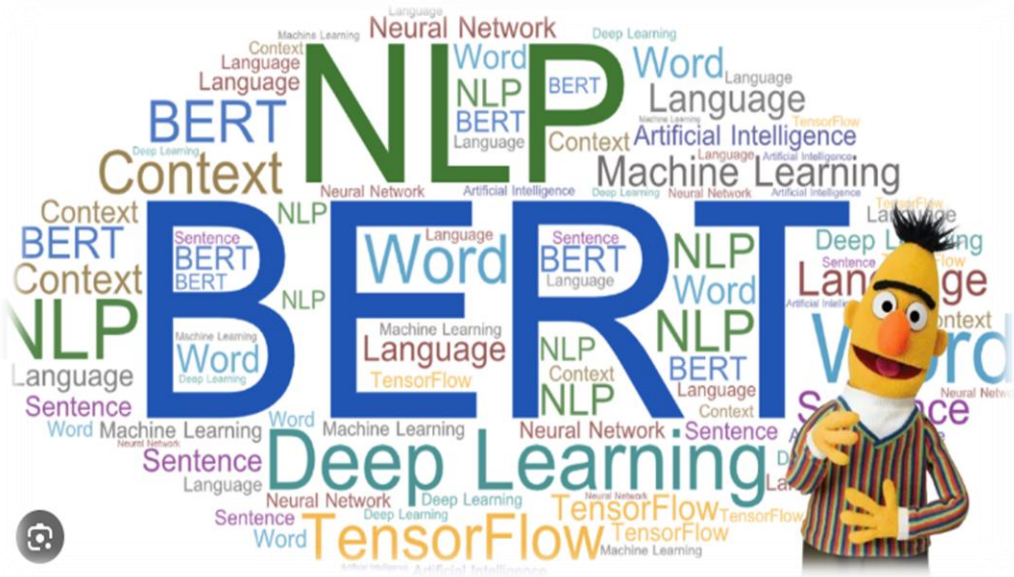
Consider these two sentences:

"I need to go to the bank to deposit a check."

"The river bank overflowed after the heavy rain."

In the first sentence, "bank" refers to a financial institution, while in the second, it means the side of a river. Static embeddings fail to capture this **polysemy** (multiple meanings of a word), making them less effective for complex NLP tasks.

BERT (Bidirectional Encoder Representations from Transformers) solves this problem by generating **contextual embeddings**. Instead of a single vector for each word, BERT creates a unique vector for a word based on its full context in a sentence. This means the vector for "bank" in the first sentence is different from the vector for "bank" in the second.



Introduction to BERT

Earlier word embedding models like Word2Vec and GloVe created a single, static vector for each word. For example, the word "bank" would always have the same vector, regardless of its context. This is a problem because words often have multiple meanings.

Consider these two sentences:

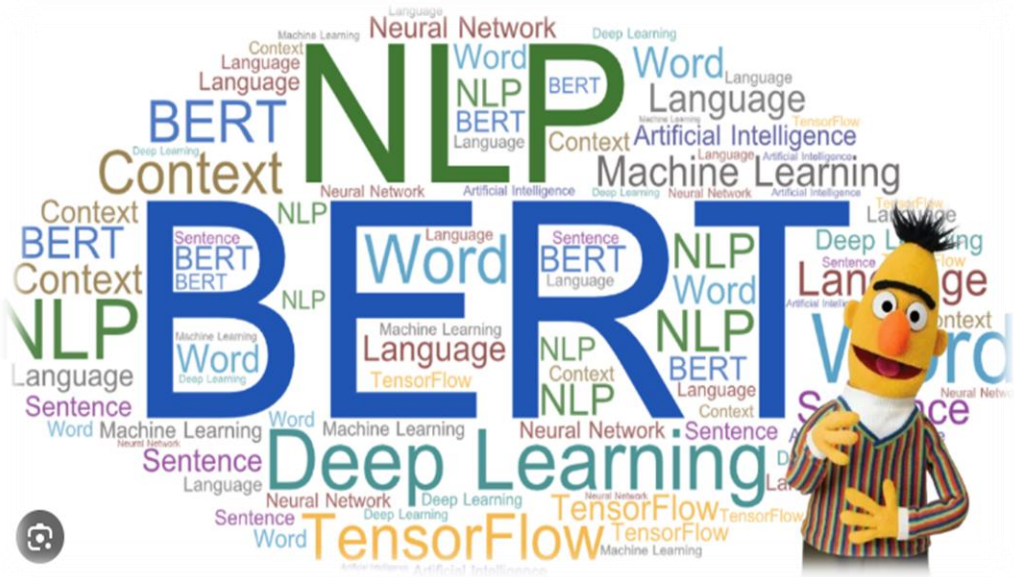
"I need to go to the bank to deposit a check."

"The river bank overflowed after the heavy rain."

In the first sentence, "bank" refers to a financial institution, while in the second, it means the side of a river. Static embeddings fail to capture this **polysemy** (multiple meanings of a word), making them less effective for complex NLP tasks.

BERT (Bidirectional Encoder Representations from Transformers) solves this problem by generating **contextual embeddings**. Instead of a single vector for each word, BERT creates a unique vector for a word based on its full context in a sentence. This means the vector for "bank" in the first sentence is different from the vector for "bank" in the second.

BERT's core innovation is its bidirectional nature. It reads a word in relation to all the other words in the sentence—both to its left and its right—at the same time. This is why it can understand the true meaning of a word in a specific context.



BERT Operational Framework

The BERT operational framework works in two distinct phases: a broad **pre-training** phase where it learns language in general, followed by a focused **fine-tuning** phase where it specializes for a specific task.

BERT Operational Framework

The BERT operational framework works in two distinct phases: a broad **pre-training** phase where it learns language in general, followed by a focused **fine-tuning** phase where it specializes for a specific task.

Phase 1: Pre-training 🧠

This is the initial, unsupervised learning phase where BERT is trained on a massive amount of unlabeled text data (like all of Wikipedia). The goal is for the model to learn the fundamental rules and patterns of language without needing specific instructions. It does this by tackling two main "fill-in-the-blank" style tasks:

BERT Operational Framework

The BERT operational framework works in two distinct phases: a broad **pre-training** phase where it learns language in general, followed by a focused **fine-tuning** phase where it specializes for a specific task.

Phase 1: Pre-training 🧠

This is the initial, unsupervised learning phase where BERT is trained on a massive amount of unlabeled text data (like all of Wikipedia). The goal is for the model to learn the fundamental rules and patterns of language without needing specific instructions. It does this by tackling two main "fill-in-the-blank" style tasks:

- **Masked Language Modeling (MLM):** The model randomly hides about 15% of the words in a sentence. Its job is to predict what the missing words are based on the words to their left and right. This forces the model to learn a deep, bidirectional understanding of context. For example, in the sentence "The quick brown [MASK] jumps over the lazy dog," BERT must learn to predict "fox."
- **Next Sentence Prediction (NSP):** The model is given two sentences, and it has to predict whether the second sentence logically follows the first one. This helps BERT understand how sentences relate to each other, which is crucial for tasks like question answering and conversation.

BERT Operational Framework

The BERT operational framework works in two distinct phases: a broad **pre-training** phase where it learns language in general, followed by a focused **fine-tuning** phase where it specializes for a specific task.

Phase 1: Pre-training 🧠

This is the initial, unsupervised learning phase where BERT is trained on a massive amount of unlabeled text data (like all of Wikipedia). The goal is for the model to learn the fundamental rules and patterns of language without needing specific instructions. It does this by tackling two main "fill-in-the-blank" style tasks:

- **Masked Language Modeling (MLM):** The model randomly hides about 15% of the words in a sentence. Its job is to predict what the missing words are based on the words to their left and right. This forces the model to learn a deep, bidirectional understanding of context. For example, in the sentence "The quick brown [MASK] jumps over the lazy dog," BERT must learn to predict "fox."
- **Next Sentence Prediction (NSP):** The model is given two sentences, and it has to predict whether the second sentence logically follows the first one. This helps BERT understand how sentences relate to each other, which is crucial for tasks like question answering and conversation.

Phase 2: Fine-Tuning 🎯

In this phase, the pre-trained BERT model is adapted for a specific, "downstream" NLP task, such as classifying text, answering questions, or summarizing documents.

BERT Operational Framework

The BERT operational framework works in two distinct phases: a broad **pre-training** phase where it learns language in general, followed by a focused **fine-tuning** phase where it specializes for a specific task.

Phase 1: Pre-training 🧠

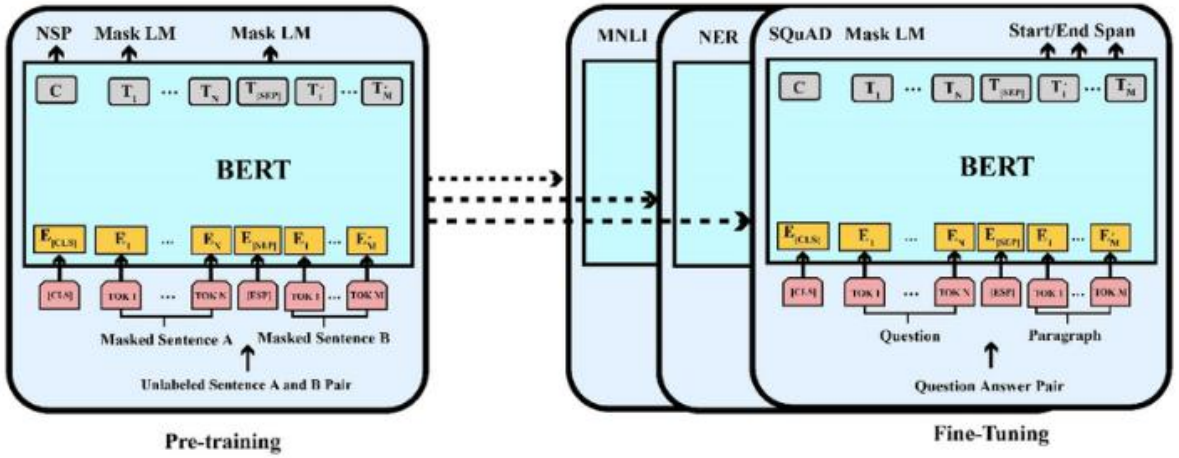
This is the initial, unsupervised learning phase where BERT is trained on a massive amount of unlabeled text data (like all of Wikipedia). The goal is for the model to learn the fundamental rules and patterns of language without needing specific instructions. It does this by tackling two main "fill-in-the-blank" style tasks:

- **Masked Language Modeling (MLM):** The model randomly hides about 15% of the words in a sentence. Its job is to predict what the missing words are based on the words to their left and right. This forces the model to learn a deep, bidirectional understanding of context. For example, in the sentence "The quick brown [MASK] jumps over the lazy dog," BERT must learn to predict "fox."
- **Next Sentence Prediction (NSP):** The model is given two sentences, and it has to predict whether the second sentence logically follows the first one. This helps BERT understand how sentences relate to each other, which is crucial for tasks like question answering and conversation.

Phase 2: Fine-Tuning 🎯

In this phase, the pre-trained BERT model is adapted for a specific, "downstream" NLP task, such as classifying text, answering questions, or summarizing documents.

- **Process:** A new, small layer is added on top of the pre-trained BERT model. This new layer is designed specifically for the new task. For example, for a sentiment analysis task, this layer might be configured to output a "positive" or "negative" score.
- **Training:** The entire model (BERT plus the new layer) is then trained on a much smaller, labeled dataset. This is a quick process because BERT already has a powerful understanding of language from the pre-training phase. It only needs to make minor adjustments to its parameters to specialize in the new task. This approach is highly efficient because you don't have to train a new model from scratch for every new problem.



Advantages and Limitation BERT

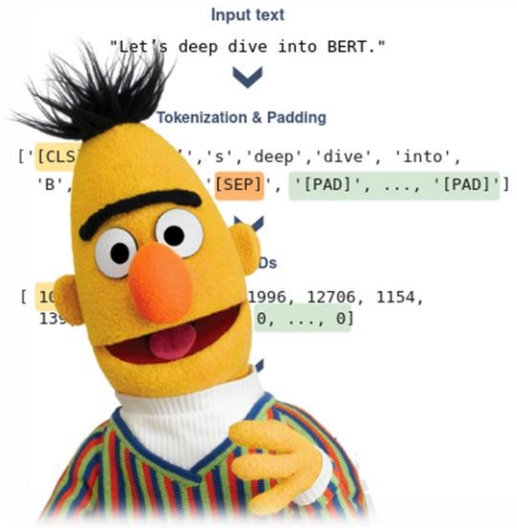
Context-Aware Embeddings: Provides a more nuanced and accurate representation of text by capturing the subtle meanings and relationships between words that traditional embeddings often miss.

State-of-the-Art Performance: Has set new industry standards by achieving top-tier performance on a wide range of NLP benchmarks and tasks, including question answering and sentiment analysis.

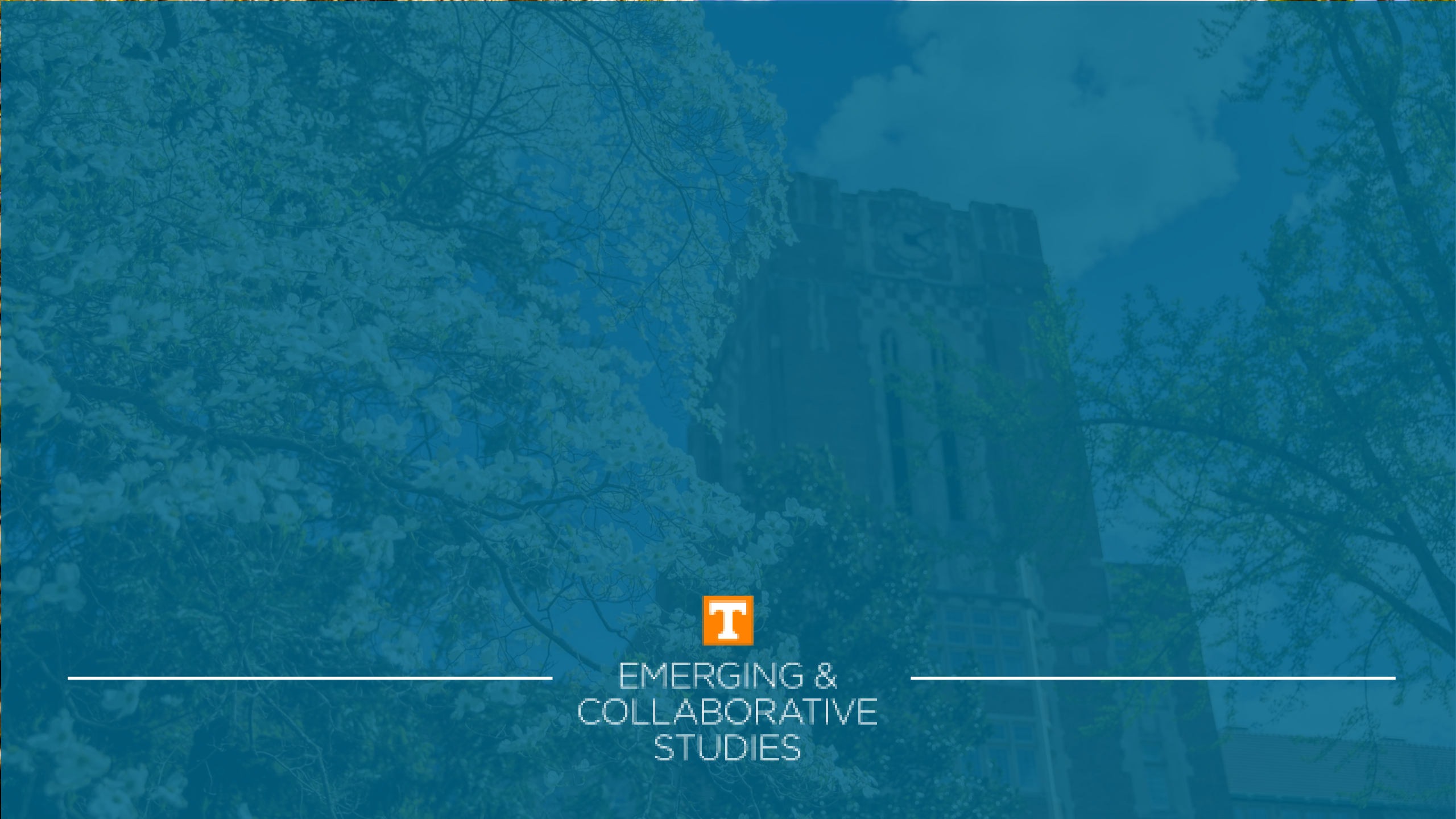
Transfer Learning: Its pre-trained models can be fine-tuned for specific tasks with relatively small amounts of labeled data, making it a highly versatile and efficient tool for various applications.

Computationally Intensive: BERT models are large and require significant computational resources for both training and inference, often necessitating access to high-performance hardware like GPUs or TPUs..

Complexity: The model's architecture and the fine-tuning process are complex, presenting a significant barrier for individuals new to NLP or those without a deep understanding of machine learning..




DEMO


EMERGING &
COLLABORATIVE
STUDIES